

Q.No:5

Question description

You have an undirected graph consisting of n vertices with weighted edges.

A simple cycle is a cycle of the graph without repeated vertices. Let the *weight* of the cycle be the **XOR** of weights of edges it consists of.

Let's say the graph is *good* if all its *simple* cycles have weight 1. A graph is bad if it's not good.

Initially, the graph is empty. Then q queries follow. Each query has the next type:

- $u\ v\ x$ — add edge between vertices u and v of weight x if it doesn't make the graph bad.

For each query print, was the edge added or not.

Constraints

$3 \leq n \leq 3 \cdot 10^5$; $1 \leq q \leq 5 \cdot 10^5$
 $1 \leq u, v \leq n$; $u \neq v$; $0 \leq x \leq 1$

Input

The first line contains two integers n and q — the number of vertices and queries.

Next q lines contain queries — one per line. Each query contains three integers u , v and x — the vertices of the edge and its weight.

It's guaranteed that there are no multiple edges in the input.

Output

For each query, print YES if the edge was added to the graph, or NO otherwise (both case-insensitive).

Logical Test Cases

Test Case 1	Test Case 2
INPUT (STDIN)	INPUT (STDIN)
9 12	9 6
6 1 0	1 3 1
1 3 1	3 6 0
3 6 0	6 4 1
6 2 0	2 4 0
6 4 1	4 5 0
3 4 1	7 8 1
2 4 0	
2 5 0	EXPECTED OUTPUT
4 5 0	YES
7 8 1	YES
8 9 1	YES
9 7 0	YES
EXPECTED OUTPUT	YES
YES	
YES	
YES	
YES	
NO	
YES	
YES	
NO	
YES	
YES	
NO	

1 KB
42 KB
35 KB
33 KB
26 KB

Code:

```
#include <stdio.h>

#define MAXV 800005

int ch[MAXV][2];
int fa[MAXV];
int revv[MAXV];
int lazy[MAXV];

int valueXor[MAXV];
int sumXor[MAXV];

int edgeCount[MAXV];
int edgeFlag[MAXV];

int badValue[MAXV];
int badSum[MAXV];

int parentDSU[MAXV];
int sizeDSU[MAXV];

int stackArr[MAXV];

int dfs1(int np, int lst)
{
    return 0;
}

int isRoot(int x)
{
    int f = fa[x];

    return f == 0 ||
    (ch[f][0] != x && ch[f][1] != x);
}

void pull(int x)
{
    int l = ch[x][0];
    int r = ch[x][1];

    sumXor[x] =
    sumXor[l] ^
    valueXor[x] ^
    sumXor[r];

    edgeCount[x] =
    edgeCount[l] +
    edgeFlag[x] +
    edgeCount[r];

    badSum[x] =
    badSum[l] +
```

```
badValue[x] +  
badSum[r];  
}  
  
void applyReverse(int x)  
{  
    if (x == 0)  
        return;  
  
    int temp = ch[x][0];  
    ch[x][0] = ch[x][1];  
    ch[x][1] = temp;  
  
    revv[x] ^= 1;  
}  
void applySet(int x)  
{  
    if (x == 0)  
        return;  
  
    badValue[x] = edgeFlag[x];  
    badSum[x] = edgeCount[x];  
  
    lazy[x] = 1;  
}  
void push(int x)  
{  
    if (revv[x])  
    {  
        applyReverse(ch[x][0]);  
        applyReverse(ch[x][1]);  
    }  
  
    revv[x] = 0;  
}  
  
if (lazy[x])  
{  
    applySet(ch[x][0]);  
    applySet(ch[x][1]);  
  
    lazy[x] = 0;  
}  
}  
void rotate(int x)  
{  
    int y = fa[x];  
    int z = fa[y];  
  
    int direction = (ch[y][1] == x);  
  
    int b = ch[x][direction ^ 1];  
  
    if (!isRoot(y))  
    {
```

```
ch[z][ch[z][1] == y] = x;
}

fa[x] = z;

ch[x][direction ^ 1] = y;
fa[y] = x;

ch[y][direction] = b;

if (b)
fa[b] = y;

pull(y);
pull(x);
}

void splay(int x)
{
int top = 0;

stackArr[top++] = x;

int y = x;

while (!isRoot(y))
{
y = fa[y];
stackArr[top++] = y;
}

while (top)
{
push(stackArr[--top]);
}

while (!isRoot(x))
{
y = fa[x];
int z = fa[y];

if (!isRoot(y))
{
if ((ch[y][0] == x) ==
(ch[z][0] == y))
{
rotate(y);
}
else
{
rotate(x);
}
}
}
```

```
rotate(x);
}
}
void accessNode(int x)
{
    int y = 0;

    while (x)
    {
        splay(x);

        ch[x][1] = y;

        pull(x);

        y = x;
        x = fa[x];
    }
}
void makeRoot(int x)
{
    accessNode(x);

    splay(x);

    applyReverse(x);
}
int findRoot(int x)
{
    accessNode(x);

    splay(x);

    push(x);

    while (ch[x][0])
    {
        x = ch[x][0];

        push(x);
    }

    splay(x);

    return x;
}

void split(int x, int y)
{
    makeRoot(x);

    accessNode(y);
```

```
splay(y);
}
void linkNodes(int x, int y)
{
    makeRoot(x);

    if (findRoot(y) != x)
    {
        fa[x] = y;
    }
}

int findDSU(int x)
{
    if (parentDSU[x] == x)
        return x;

    parentDSU[x] =
        findDSU(parentDSU[x]);

    return parentDSU[x];
}

void unionDSU(int a, int b)
{
    a = findDSU(a);
    b = findDSU(b);

    if (a == b)
        return;

    if (sizeDSU[a] < sizeDSU[b])
    {
        int temp = a;
        a = b;
        b = temp;
    }

    parentDSU[b] = a;

    sizeDSU[a] += sizeDSU[b];
}

int main()
{
    int n, q;

    scanf("%d %d", &n, &q);

    int total = n + q;

    for (int i = 1; i <= total; i++)
    {
```

```
parentDSU[i] = i;
sizeDSU[i] = 1;
}

for (int i = 1; i <= n; i++)
{
    pull(i);
}
for (int i = 1; i <= q; i++)
{
    int u, v, x;

    scanf("%d %d %d", &u, &v, &x);

    if (findDSU(u) != findDSU(v))
    {
        int edgeNode = n + i;

        edgeFlag[edgeNode] = 1;

        valueXor[edgeNode] = x;

        pull(edgeNode);

        linkNodes(u, edgeNode);

        linkNodes(edgeNode, v);

        unionDSU(u, v);

        printf("YES\n");
    }
    else
    {
        split(u, v);
        if (badSum[v] == 0 &&
            (sumXor[v] ^ x) == 1)
        {
            applySet(v);

            printf("YES\n");
        }
        else
        {
            printf("NO\n");
        }
    }
}
```

```
return 0;  
}
```

Output:

Test_case 1:

```
aadityajaiswal@Aadityas-Ma  
9 12  
6 1 0  
YES  
1 3 1  
YES  
3 6 0  
YES  
6 2 0  
YES  
6 4 1  
YES  
3 4 1  
NO  
2 4 0  
YES  
2 5 0  
YES  
4 5 0  
NO  
7 8 1  
YES  
8 9 1  
YES  
9 7 0  
NO  
aadityajaiswal@Aadityas-Ma
```

Test_case 2:

```
aadityajaiswa  
9 6  
1 3 1  
YES  
3 6 0  
YES  
6 4 1  
YES  
2 4 0  
YES  
4 5 0  
YES  
7 8 1  
YES  
aadityajaiswa
```